

Chapter 15

Introduction to Agentic AI

The previous parts equipped us with the algorithmic toolkit—how to train, align, and reason with LLMs. We covered transformer architectures and GPU systems (Part I), the reinforcement learning methods that align models with human intent (Part II), the reasoning capabilities that emerge from RL training (Part III), and evaluation methodology (Part IV). This part turns to the central question of modern AI engineering: how do we *deploy* these models as autonomous agents that perceive, plan, act, and learn in open-ended environments?

An **agentic AI system** is one where an LLM operates in a loop: it receives observations from an environment (user messages, tool outputs, sensor data), reasons about what to do next, takes actions (tool calls, code execution, API requests), and iterates until a goal is achieved or it explicitly asks for human input. This contrasts with the “single-turn chatbot” paradigm where the model produces one response and waits.

The shift from chatbot to agent introduces several fundamental challenges that a single model call cannot address:

- **Persistence:** An agent must remember what it has done, what failed, and what context was established—across turns, sessions, and even days.
- **Grounding:** The agent must access up-to-date, domain-specific knowledge that was not present in its training data.
- **Action:** The agent must interact with external systems—databases, APIs, file systems, browsers—through well-defined interfaces.
- **Coordination:** Complex tasks often exceed what a single agent can handle; multiple specialized agents must collaborate, delegate, and negotiate.
- **Safety:** Autonomous action requires guardrails, human oversight, and graceful degradation when the agent is uncertain.

To address these challenges, production agentic systems are built as a layered architecture. Each layer solves a specific problem, and the chapters that follow cover the full stack from bottom to top:

- **Chapter 16: RAG (Retrieval-Augmented Generation)** — The knowledge layer. RAG gives agents access to dynamic external knowledge by retrieving relevant documents at query time. This solves the grounding problem: agents can answer questions about proprietary data, recent events, or domain-specific content that the model never saw during training. We cover embedding models, vector databases, chunking strategies, hybrid retrieval, and advanced patterns like agentic RAG where the agent decides *when* and *what* to retrieve.
- **Chapter 17: Memory** — The persistence layer. Memory enables agents to recall information across interactions—from short-term working memory within a single task, to long-term episodic memory spanning months. We cover memory architectures (buffer, summary, vector-indexed, knowledge graphs), memory consolidation, and how to design memory systems that scale without drowning the context window.

- **Chapter 18: Harness & Orchestration** — The runtime layer. The orchestration harness is the “operating system” for agents: it manages the agent loop, context window budget, tool dispatch, error recovery, state persistence, and observability. We cover context management strategies (summarization, sliding window, hierarchical), execution control (sequential, parallel, branching), guardrails, and human-in-the-loop patterns.
- **Chapter 19: Loop Engineering** — The inference-time RL layer. Loop engineering treats the agent’s runtime loop as an optimization process: generate–verify–retry cycles that are structurally equivalent to policy optimization without weight updates. We cover the five structural primitives (generator, verifier, terminator, state manager, escalator), verification hierarchies, adaptive budget allocation, and how modern loops evolved from early AutoGPT-style systems into disciplined convergent processes.
- **Chapter 20: Design Patterns** — The architecture layer. Canonical patterns for structuring agents: ReAct (reason + act interleaving), plan-then-execute, reflection loops, tool-augmented generation, and multi-step workflows. We analyze when each pattern applies, their failure modes, and how to combine them for complex real-world tasks.
- **Chapter 21: Environments & Benchmarks** — The evaluation layer. Where and how to evaluate agentic behaviour. We cover web navigation benchmarks, coding environments, tool-use evaluation suites, and the unique challenges of evaluating multi-step autonomous systems (partial credit, trajectory quality, safety violations).
- **Chapter 22: MCP (Model Context Protocol)** — The tool integration standard. MCP standardizes how agents discover and invoke tools—analogue to USB for hardware. We cover the protocol specification, server/client architecture, resource management, and how MCP eliminates the N×M integration problem between agents and tools.
- **Chapter 23: Agent Skills** — The capability layer. How agents acquire and compose specialized capabilities beyond basic tool use, including skill libraries, skill selection, and compositional task solving.
- **Chapter 24: A2A (Agent-to-Agent Communication)** — The inter-agent protocol. When tasks require multiple specialists, A2A provides a standardized protocol for agent discovery, task delegation, progress streaming, and result aggregation—enabling heterogeneous agents (from different vendors, frameworks, or organizations) to collaborate.
- **Chapter 25: Multi-Agent Systems** — The coordination layer. Architectures for multi-agent collaboration: hierarchical delegation, peer-to-peer negotiation, debate and consensus, swarm intelligence, and emergent behaviour. We cover when to use single-agent vs. multi-agent designs and how to debug coordination failures.
- **Chapter 26: Frameworks** — The implementation layer. Production toolkits that implement the above concepts: LangGraph (stateful graph-based orchestration), CrewAI (role-based multi-agent), OpenAI Agents SDK, AutoGen, and others. We compare their trade-offs, architecture decisions, and suitability for different use cases.
- **Chapter 27: Agentic UI** — The interaction layer. How users interact with and supervise agents: streaming interfaces, progressive disclosure, approval workflows, status dashboards, and the UX patterns that build appropriate trust in autonomous systems.

These layers do not operate in isolation—they form a tightly integrated system where each component depends on and enhances the others:

- The **agent core** (an LLM with reasoning capabilities from Parts II–III) sits at the center, executing a perceive–reason–act loop.

- **RAG** feeds the agent with relevant knowledge before each reasoning step, while **Memory** provides continuity across steps and sessions.
- The **Orchestration Harness** coordinates everything: it decides when to retrieve, when to call tools, when to delegate to sub-agents, and when to ask the human for guidance. **Loop Engineering** formalizes the harness's retry-and-verify cycles as inference-time optimization, ensuring convergence and cost control.
- **MCP** provides the standardized interface through which the agent accesses all external tools, and **A2A** provides the equivalent interface for inter-agent communication.
- **Design Patterns** define the high-level strategy (ReAct, plan-and-execute, reflection), while **Frameworks** provide the concrete implementation of these patterns.
- The **UI layer** closes the loop by connecting the agent back to the human—for oversight, correction, and collaborative problem-solving.

Throughout, we maintain the systems perspective: agentic AI is not just about prompting—it requires careful engineering of context management, error handling, safety guardrails, and observability at every layer. The figure below shows how these components fit together architecturally.

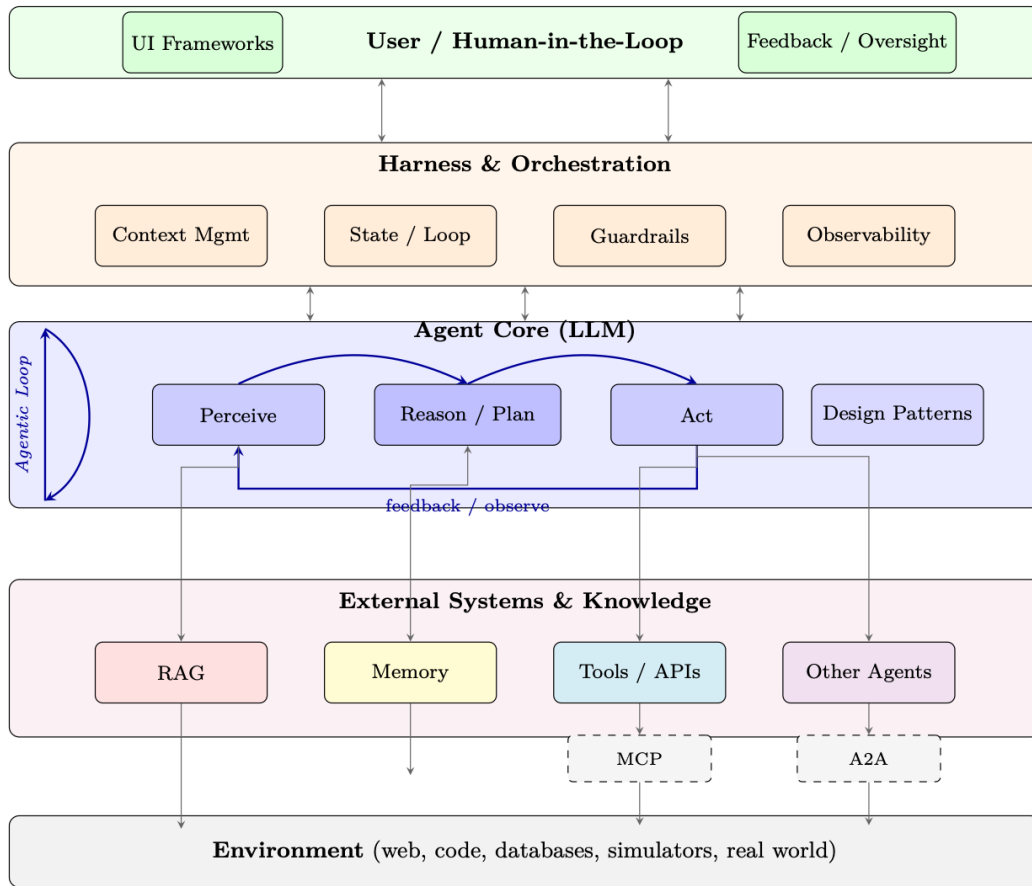


Figure 15.1: The Agentic AI architecture stack. The **Agent Core** executes a perceive–reason–act loop, coordinated by the **Harness & Orchestration** layer which manages context, state, guardrails, and observability. The agent interacts downward with **External Systems**—RAG for knowledge retrieval, Memory for persistence, Tools via MCP, and other Agents via A2A—all grounded in an **Environment**. The **User** provides goals, feedback, and oversight from above. Arrows indicate bidirectional data flow; the blue loop arrows show the iterative agentic cycle.